

Database Assignment 2

Name: Habiba Saber

Part 1: Transactions

1. Write a SQL transaction to: Insert a new student, insert their course enrollment, if enrollment fails → rollback everything.

```
BEGIN TRANSACTION;

BEGIN TRY
    INSERT INTO students (id, name) VALUES (101, 'Ahmed');
    INSERT INTO enrollments (student_id, course_id) VALUES (101, 5);
    COMMIT;
END TRY
BEGIN CATCH
    ROLLBACK;
END CATCH;
```

2. Explain the difference between COMMIT vs ROLLBACK. Give a practical example.

COMMIT: Saves all transaction changes permanently to the database.

ROLLBACK: Undoes all transaction changes, returning the database to its previous state.

Example: Transferring money. If you deduct \$100 from Account A, but the system crashes before adding it to Account B, a ROLLBACK ensures the \$100 is returned to A. If both succeed, a COMMIT saves the transfer.

3. Write a transaction that: Deducts 500 from Account A, Adds 500 to Account B, Ensure atomicity.

```
BEGIN TRANSACTION;

UPDATE accounts SET balance = balance - 500 WHERE account_name = 'A';
UPDATE accounts SET balance = balance + 500 WHERE account_name = 'B';

COMMIT;
```

4. What is ACID property? Explain each with example.

A - Atomicity: All or nothing. (e.g., A money transfer either fully completes or completely fails).

C - Consistency: Database remains in a valid state before and after. (e.g., Account balances cannot drop below zero if a constraint exists).

I - Isolation: Concurrent transactions don't interfere. (e.g., Two people withdrawing from the same ATM won't read the same balance incorrectly).

D - Durability: Committed data is saved permanently, even after power loss. (e.g., Database saves to physical disk immediately upon commit).

5. Write SQL using SAVEPOINT: Insert 3 records, Rollback only the second insert.

```
BEGIN TRANSACTION;
INSERT INTO students (id, name) VALUES (1, 'Ali');

SAVEPOINT sp1;
INSERT INTO students (id, name) VALUES (2, 'Omar');

ROLLBACK TO sp1; -- Undoes Omar

INSERT INTO students (id, name) VALUES (3, 'Hassan');
COMMIT;
```

Part 2: Indexes

6. Create an index on students(email) and explain why it improves performance.

```
CREATE INDEX idx_student_email ON students(email);
```

It improves performance by creating a B-Tree structure for the email column, allowing the database engine to find specific emails logarithmically without scanning every single row in the table (Full Table Scan).

7. What is the difference between Clustered and Non-Clustered Index?

Clustered Index: Dictates the physical storage order of the data rows. A table can only have one clustered index (usually the Primary Key).

Non-Clustered Index: A separate structure from the data rows that contains pointers to the actual data. A table can have multiple non-clustered indexes.

8. Write SQL to: Create composite index on (first_name, last_name).

```
CREATE INDEX idx_student_name ON students(first_name, last_name);
```

9. When should you NOT use indexes? Give 2 real scenarios.

1. **Very small tables:** Scanning the whole table is faster than looking up the index tree.
2. **Tables with frequent write operations:** In tables with massive bulk INSERTs or UPDATEs (like log tables), indexes slow down performance because the index tree must be constantly rebuilt/updated.

10. Analyze this query and suggest indexing: SELECT * FROM orders WHERE customer_id = 5 AND order_date = '2024-01-01';

A composite index should be created covering both columns used in the WHERE clause to optimize the lookup.

```
CREATE INDEX idx_customer_order_date ON orders(customer_id, order_date);
```

Part 3: Joins

11. Write INNER JOIN to get students with their courses.

```
SELECT s.name, c.title
FROM students s
INNER JOIN enrollments e ON s.id = e.student_id
INNER JOIN courses c ON e.course_id = c.id;
```

12. Write LEFT JOIN to show all students even if not enrolled.

```
SELECT s.name, c.title
FROM students s
LEFT JOIN enrollments e ON s.id = e.student_id
LEFT JOIN courses c ON e.course_id = c.id;
```

13. Write RIGHT JOIN example and explain when it's used.

```
SELECT s.name, c.title
FROM enrollments e
RIGHT JOIN courses c ON e.course_id = c.id;
```

Usage: Used here to find all available courses, including those that have zero students enrolled in them.

14. Write a query to find students NOT enrolled in any course.

```
SELECT s.name
FROM students s
LEFT JOIN enrollments e ON s.id = e.student_id
WHERE e.course_id IS NULL;
```

15. Write a query to count number of courses per student.

```
SELECT s.name, COUNT(e.course_id) as course_count
FROM students s
LEFT JOIN enrollments e ON s.id = e.student_id
GROUP BY s.id, s.name;
```

Part 4: Subqueries

16. Write a subquery to find students who scored above average marks.

```
SELECT name
FROM students
WHERE marks > (SELECT AVG(marks) FROM students);
```

17. Write a query using IN subquery: Get students enrolled in course "Database".

```
SELECT name
FROM students
WHERE id IN (
    SELECT student_id
    FROM enrollments
    WHERE course_id IN (
        SELECT id FROM courses WHERE title = 'Database'
    )
);
```

18. Write a correlated subquery: Find students whose marks are higher than their department average.

```
SELECT name, marks
FROM students s1
WHERE marks > (
    SELECT AVG(marks)
    FROM students s2
    WHERE s1.department_id = s2.department_id
);
```

19. Rewrite this using JOIN instead of subquery.

```
SELECT DISTINCT s.name
FROM students s
INNER JOIN enrollments e ON s.id = e.student_id;
```

20. Write a query using EXISTS: Find students who are enrolled in at least one course.

```
SELECT name
FROM students s
WHERE EXISTS (
    SELECT 1
    FROM enrollments e
    WHERE e.student_id = s.id
);
```